

uFldNodeBroker, uFldShoreBroker, and uFldNodeComms

uField broker and node communication tools.

Included MIT MOOS-IvP PDF sources:

- app_ufld_nodebroker.pdf
- app_ufld_shorebroker.pdf
- app_ufld_nodecomms.pdf

uFldNodeBroker: Brokering Node Connections

June 2018

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139

1	Overview	1
2	The uFldNodeBroker Interface and Configuration Options	2
2.1	Configuration Parameters of uFldNodeBroker	2
3	Publications and Subscriptions for uFldNodeBroker	4
3.1	Variables Published by uFldNodeBroker	4
3.2	Variables Subscribed for by uFldNodeBroker	4
3.3	Command Line Usage of uFldNodeBroker	4
4	Terminal and AppCast Output	5

1 Overview

The `uFldNodeBroker` application is a tool for brokering connections between a node (a simulated or real vehicle) and a shoreside community. It is used primarily in coordination with `uFldShoreBroker` to discover and share host IP and port information to automate the dynamic configurations of `pShare`. Inter-vehicle communications over the network are handled by `pShare` in both simulation with single or multiple machines as well as on fielded vehicles using Wi-Fi or cellphone connections. The `pShare` application simply needs to know the IP address and port number of connected machines. Often these aren't known at run-time and even if they were, maintaining that information in configuration files may be cumbersome, especially for large sets of vehicles. This tool is meant to automate the configuration by letting the nodes and shoreside community discover each other by giving the node (`uFldNodeBroker`) some initial hints on where to find the shoreside community on the network. The typical layout is shown in Figure 1.

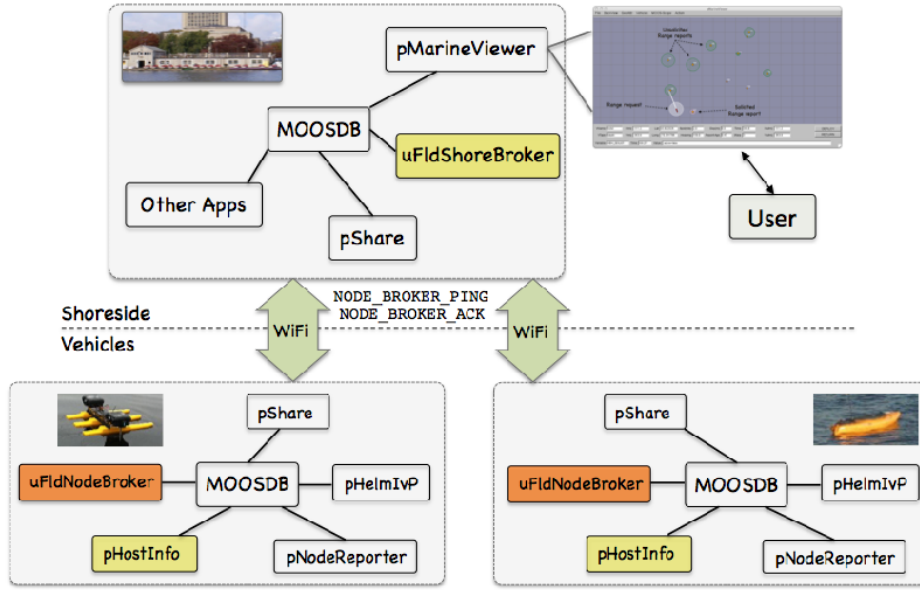


Figure 1: **Typical uFldNodeBroker Topology:** A vehicle (node) sends information about itself (IP address and port number) to possible shoreside locations. Once a connection is made, further sharing is established between the node and shoreside communities.

The functionality of `uFldNodeBroker` paraphrased:

- Discover the node's host information (typically from `pHostInfo`).
- For candidate shoreside hosts, request a new share configuration in `pShare` to each candidate for the variable `NODE_BROKER_PING`.
- Publish `NODE_BROKER_PING` with the node's host information.
- Await a reply in the form of incoming `NODE_BROKER_ACK` mail, presumably from the shoreside community running `uFldShoreBroker`.
- Now that a shoreside community is known, request new share configurations from the node's local `pShare` for all the info otherwise wanted for sharing to the shoreside.
- Keep sending pings periodically in case the shoreside community is re-started and needs to re-establish connections to nodes.

2 The uFldNodeBroker Interface and Configuration Options

The `uFldNodeBroker` application may be configured with a configuration block within a `.moos` file. Its interface is defined by its publications and subscriptions for MOOS variables consumed and generated by other MOOS applications. An overview of the set of configuration options and interface is provided in this section. If one has access to a command line where `uFldNodeBroker` has been built, interface information may also be seen by typing "`uFldNodeBroker --interface`", and configuration information by typing "`uFldNodeBroker --example`".

2.1 Configuration Parameters of uFldNodeBroker

The following parameters are defined for `uFldNodeBroker`.

Listing 2.1: Configuration Parameters for uFldNodeBroker.

<code>auto_bridge_appcast:</code>	Suppress the normal automatic bridging of the <code>APPCAST</code> variable. The default is false.
<code>auto_bridge_realmcas</code>	Suppress the normal automatic bridging of the <code>REALMCAST</code> and <code>REALMCAST_CHANNELS</code> variables. The default is false.
<code>bridge:</code>	A variable to register with <code>pShare</code> for bridging to a shoreside MOOS community once a shoreside connection has been established.
<code>try_shore_host:</code>	A candidate route to send initial pings in hopes of establishing a connection. The route specifies an input route for pShare running in a shoreside community.

Since appcasting and realmcasting are common and recommended practices for users of the uField Toolbox, key variables that are bridged are done automatically. Normally the `APPCAST` variable is auto bridged to support appcasting. And the `REALMCAST` and `REALMCAST_CHANNELS` variables are auto bridged to support realmcasting. The user has the option to opt out of this configuration by setting the `auto_bridge.*` configuration parameters described above. Note that realmcasting was first introduced to the uField Toolbox and MOOS-IvP code in the first release after 19.8.1.

An Example MOOS Configuration Block

An example MOOS configuration block can be obtained by entering the following from the command-line:

```
$ uFldNodeBroker --example or -e
```

Listing 2.2: Example configuration of the uFldNodeBroker application.

```
1  =====
2  uFldNodeBroker Example MOOS Configuration
3  =====
4
5  ProcessConfig = uFldNodeBroker
6  {
7      AppTick    = 4
8      CommsTick  = 4
9
10     keyword      = lemon
11
12     try_shore_host = pshare_route=localhost:9200
13     try_shore_host = pshare_route=192.168.0.122:9301
14     try_shore_host = pshare_route=multicast_8
15
16     bridge = src=VIEW_POLYGON
17     bridge = src=VIEW_POINT
18     bridge = src=VIEW_SEGLIST
19
20     bridge = src=NODE_REPORT_LOCAL, alias=NODE_REPORT
21 }
```

3 Publications and Subscriptions for uFldNodeBroker

The interface for `uFldNodeBroker`, in terms of publications and subscriptions, is described below. This same information may also be obtained from the terminal with:

```
$ uFldNodeBroker --interface or -i
```

3.1 Variables Published by uFldNodeBroker

The primary output of `uFldNodeBroker` to the MOOSDB are the requests to `pShare` for registrations, and the outgoing pings to candidate shoreside communities.

- **APPCAST**: Contains an appcast report identical to the terminal output. Appcasts are posted only after an appcast request is received from an appcast viewing utility. Section 4.
- **NODE_BROKER_PING**: A message written locally but bridged to a candidate shoreside MOOS community, containing IP address and `pShare` route information about the local community.
- **PSHARE_CMD**: message to `pShare` to add a new bridge for a given variable and given target MOOS community at a specified IP address and port number.

3.2 Variables Subscribed for by uFldNodeBroker

The `uFldNodeBroker` application subscribes to the following MOOS variables:

- **APPCAST_REQ**: A request to generate and post a new apppcast report, with reporting criteria, and expiration.
- **NODE_BROKER_ACK**: Information published presumably by `uFldShoreBroker` running in a separate shoreside community. Message has information about the shoreside host including the community name, IP address and port numbers for the MOOSDB and its local `pShare` process.
- **PHI_HOST_INFO**: Information about the local host IP address, the MOOS community name, the port on which the DB is running, and the port on which the local `pShare` is listening for UDP messages.

3.3 Command Line Usage of uFldNodeBroker

The `uFldNodeBroker` application is typically launched with `pAntler`, along with a group of other shoreside modules. However, it may be launched separately from the command line. The command line options may be shown by typing

```
$ uFldNodeBroker --help or -h
```

Listing 3.3: Command line usage for the `uFldNodeBroker` tool.

```
1 =====
2 Usage: uFldNodeBroker file.moos [OPTIONS]
3 =====
4
```

```

5 Options:
6   --alias=<ProcessName>
7       Launch uFldNodeBroker with the given
8       process name rather than uFldNodeBroker.
9   --example, -e
10      Display example MOOS configuration block.
11   --help, -h
12      Display this help message.
13   --interface, -i
14      Display MOOS publications and subscriptions.
15   --version, -v
16      Display release version of uFldNodeBroker.

```

4 Terminal and AppCast Output

The `uFldNodeBroker` application produces some useful information to the terminal on every iteration of the application. An example is shown in Listing 4 below. This application is also appcast enabled, meaning its reports are published to the MOOSDB and viewable from any uMAC application or `pMarineViewer`. The counter on the end of line 2 is incremented on each iteration of `uFldNodeBroker`, and serves a bit as a heartbeat indicator. The "0/0" also on line 2 indicates there are no configuration or run warnings detected.

Listing 4.4: Example terminal or appcast output for `uFldNodeBroker`.

```

1  =====
2  uFldNodeBroker henry                                0/0(129)
3  =====
4
5  Total OK PHI_HOST_INFO      received: 13
6  Total BAD PHI_HOST_INFO     received: 0
7  Total HOST_INFO changes     received: 1
8  Total          PSHARE_CMD    posted: 7
9  Total BAD NODE_BROKER_ACK   received: 6
10
11 =====
12          Vehicle Node Information:
13 =====
14
15     Community: henry
16     HostIP: 10.0.0.5
17     Port MOOSDB: 9001
18     Time Warp: 4
19     IRoutes: 10.0.0.5:9301
20
21 =====
22          Shoreside Node(s) Information:
23 =====6
24
25 Community      Pings  Pings  IP      Time
26 Name          Route   Sent   Acked   Address  Warp
27 -----
28 shoreside     localhost:9300 128    120    10.0.0.5  4
29 Phase Completion Summary:
30 -----

```

```

31 Phase 1: (Y) Valid Host information retrieved (iroutes).
32 Phase 2: (Y) Valid TryHosts (1) configured.
33 Phase 3: (Y) NODE_BROKER_PINGS are being sent to TryHosts.
34 Phase 4: (Y) A Valid NODE_BROKER_ACK has been received.
35 Phase 5: (Y) pShare requested to share user vars with shoreside.
36 All Phases complete. Things should be working as configured.

```

On line 5, the number of incoming mail messages for **PHI.HOST.INFO** is tallied where the host information bundle is deemed complete. It is complete if it contains the host community, IP address, time warp and **pShare** input route information. If an incomplete host information packet is received, it is tallied in line 6. The number on line 6 should always be zero. If the number on line 6 is not zero, or the number on line 5 never increments, you should check the operation of the **pHostInfo**.

The number on line 7 indicates the number of time the detected host information changes. This number should stabilize very quickly maxing out at 1 or 2 typically. The number on line 8 indicates the number of dynamic bridge requests posted to **pShare**. These are in the form of postings to the variable **PSHARE.CMD**, which occur first for outgoing pings to candidate shoreside hosts, and then for the user **bridge** configuration variables after a shoreside host has been connected. Invalid **NODE.BROKER.ACK** messages are tallied on line 9. An invalid ack may be due to a mismatch in time warp between node and shoreside, or a mismatch in keywords if keywords are being used.

Self node information is displayed in the next group, lines 11-19. The community name, MOOSDB port, and time warp are all read from the .moos mission configuration file. The node's IP address (line 16) and the local **pShare** input routes (line 19) are obtained from output received from **pHostInfo**.

Information about candidate and connected shoreside communities is shown next in lines 21-28. In this case there is only one entry, line 27. For each candidate entry, the community name, shoreside **pShare** input route, number of pings sent and acknowledged are in the first four columns. The last two columns indicate the IP address and time warp of the shoreside learned from **NODE.BROKER.ACK** messages received from the shoreside. When a candidate shoreside community has *not* been connected, the entry in this table will something like:

Community Name	Route	Pings Sent	Pings Aked	IP Address	Time Warp
-----	-----	-----	-----	-----	----
	localhost:92003	385	0		

The last block of information in the report is the Phase Completion Summary, lines 29-36. It lists the rough sequence of events typical to reach a shoreside community connection. If any one of these phases is incomplete, the output on line 36 will be replaced with a few hints on where to troubleshoot.

uFldShoreBroker: Brokering Shore Connections

June 2018

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139

1	Overview	1
2	Bridging Variables Upon Connection to Nodes	2
2.1	Inter-MOOSDB Bridging with pShare	2
2.2	Handling a Valid Incoming Ping from a Remote Node	3
2.3	Vanilla Bridge Arrangements	4
2.4	Bridge Arrangements with Macros	4
2.5	A Common Configuration Shortcut - the qbridge Parameter	5
3	Usage Scenarios for the uFldShoreBroker Utility	5
4	Terminal and AppCast Output	6
5	Configuration Parameters of uFldShoreBroker	7
6	Publications and Subscriptions for uFldShoreBroker	8
6.1	Variables Published by uFldShoreBroker	9
6.2	MOOS Variables Subscribed for by uFldShoreBroker	9
6.3	Command Line Usage of uFldShoreBroker	9

1 Overview

The **uFldShoreBroker** application is a tool for brokering connections between a shoreside community and one or more nodes (simulated or real vehicles). A shoreside community is collection of MOOS processes typically running a GUI providing a situational display and managing messages to and from fielded vehicles. This is depicted in the notional rendering in Figure 1 below. The shoreside community in practice is often situated on a ship with UUVs below, and is more aptly referred to as the topside community. The user interacts with the GUI or perhaps other communication modules, to send high-level messages to the vehicles.

The **uFldShoreBroker** application is used primarily in coordination with **uFldNodeBroker**, running on the vehicles, to discover and share host IP and port information to automate the dynamic configurations of **pShare**. Inter-vehicle communications over the network are handled by **pShare** in both simulation with single or multiple machines as well as on fielded vehicles using Wi-Fi or cellphone connections. The **pShare** application simply needs to know the IP address and port number of connected machines. Often these aren't known at run-time and even if they were, maintaining that information in configuration files may be unduly cumbersome, especially for large sets of vehicles. This tool is meant to automate the configuration by letting the nodes and shoreside community discover each other by letting (**uFldShoreBroker**) respond to incoming pings, i.e., initialization messages, from nodes on the network. The typical layout is shown in Figure 1.

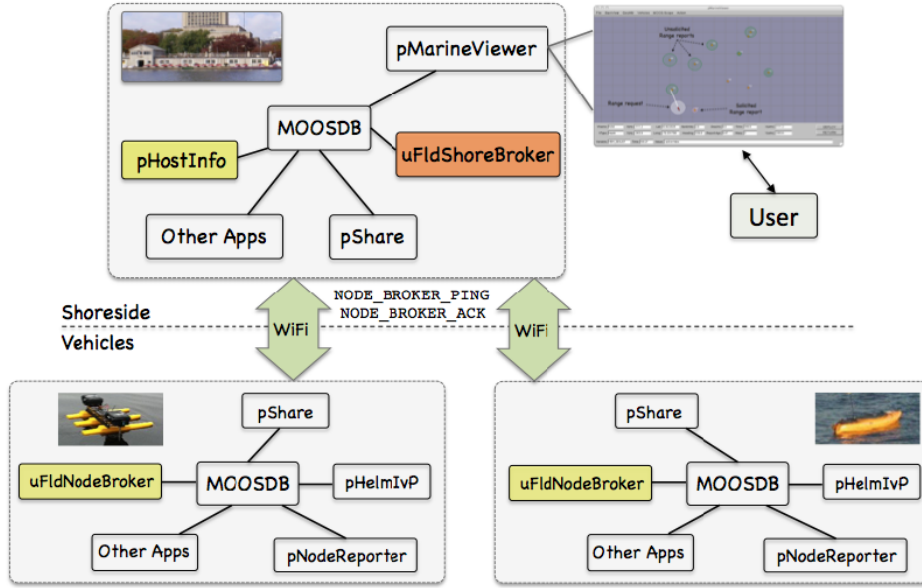


Figure 1: **Typical `uFldShoreBroker` Topology:** A vehicle (node) sends information about itself (IP address and port number) to the shoreside, received by `uFldShoreBroker`. It responds by (a) acknowledging the connection to the node, and (b) establishing user configured bridges of particular MOOS variables to the node.

The functionality of `uFldShoreBroker` paraphrased:

- Discover the shoreside's own host information (typically from `pHostInfo`).
- Await incoming `NODE_BROKER_PING` messages from non-local vehicles.
- Upon an incoming ping, respond to the nodes with a `NODE_BROKER_ACK` message to the location specified in the ping message.
- Establish new bridges to the nodes for variables specified previously by the user in the `uFldShoreBroker` configuration.
- Keep sending acknowledgments periodically to confirm to vehicles that they are still connected to the shoreside community.

2 Bridging Variables Upon Connection to Nodes

A primary function of `uFldShoreBroker` is to establish bridging relationships to a remote node community after it has received a ping from that community. These variables are specified in the configuration block with lines like `bridge = "src=DEPLOY_ALL, alias=DEPLOY"` as in Listing 3. This step is described next.

2.1 Inter-MOOSDB Bridging with `pShare`

Static bridging with `pShare` is done by specifying the desired route in the `pShare` configuration block with a line of the form:

```
output = src_name=VAR, dest_name=ALIAS, route=ROUTE
```

For example:

```
output = src_name=DEPLOY_HENRY, dest_name=DEPLOY, route=12.56.111.1:9200
```

The above connection may be used to send the vehicle Henry the deploy command from the shoreside community. The problem is that the shoreside may not know the IP address of Henry (or the port on which it's pShare is listening) until it presents itself to the shoreside at run time.

In this case, dynamic share registration needs to be used by sending **pShare** a message after it has been launched. For example, the above sharing relationship could be established by sending the following message:

```
PSHARE_CMD = "cmd=output, src_name=DEPLOY_ALL, dest_name=DEPLOY,  
route=2.56.111.1:9200
```

It is the job of **uFldShoreBroker** to post the above style dynamic requests once the node information becomes known to the shoreside community.

2.2 Handling a Valid Incoming Ping from a Remote Node

The basic job of **uFldShoreBroker** is to await incoming pings, and use the information in a ping message to (a) decide if the ping should be accepted, and (b) send the appropriate response back to the sender, and (c) set up new outgoing **pShare** relationships if the ping is indeed accepted. The contents of a ping may look something like:

```
NODE_BROKER_PING = "community=henry,host=192.168.1.22,port=9000,time_warp=10  
pshare_iroutes=192.168.1.22:9200,time=1325178800.81"
```

There must be a match in the MOOS *time warp* used by the shoreside MOOS community and any node connected to the shore. This is always 1 when operating vehicles in the field, but may be set to a much larger number in simulation. The time warp is set with the parameter **MOOSTimeWarp** at the top of the .moos configuration file.

The ping consists of three key pieces of information:

- The community name of the node,
- The IP address of the node,
- The input routes on which the node is listening for messages with its own local **pShare** running.

Once this information is known by the shoreside broker, new bridges can be established for variables identified by the user. The only other information needed is (a) the name of the variable in the local **MOOSDB**, and (b) the name (alias) of the variable as it is to be known in the remote **MOOSDB**. Once a valid ping has been received and accepted, **uFldShoreBroker** is ready to establish bridge arrangements with its local pShare running.

2.3 Vanilla Bridge Arrangements

The simplest bridge arrangement specifies (a) the variable as it is known locally, and (b) the variable name as it is to be known remotely. This is done with a `uFldShoreBroker` configuration line similar to:

```
bridge = src=DEPLOY_ALL, alias=DEPLOY
```

For *each* unique incoming ping, a new bridge arrangement will be requested. By unique, we mean having a distinct community (remote vehicle) name. For example, if pings are received and accepted from *henry*, *james*, and *ike*, `uFldShoreBroker` would make three separate posts, perhaps looking like:

```
PSHARE_CMD = "src_name=DEPLOY_ALL, dest_name=DEPLOY, route=2.56.111.1:9200"  
PSHARE_CMD = "src_name=DEPLOY_ALL, dest_name=DEPLOY, route=2.56.111.3:9200"  
PSHARE_CMD = "src_name=DEPLOY_ALL, dest_name=DEPLOY, route=2.56.111.6:9200"
```

At this point the behavior of `pShare` on the shoreside would be functionally equivalent to the scenario where the following three lines were in the `pShare` configuration block:

```
output = src_name=DEPLOY_ALL, dest_name=DEPLOY, route=12.56.111.1:9200  
output = src_name=DEPLOY_ALL, dest_name=DEPLOY, route=12.56.111.3:9200  
output = src_name=DEPLOY_ALL, dest_name=DEPLOY, route=12.56.111.6:9200
```

2.4 Bridge Arrangements with Macros

The user may configure `uFldShoreBroker` with bridge arrangements containing a couple types of macros. For example, consider the configuration:

```
bridge = src=DEPLOY_$V, alias=DEPLOY
```

The `$V` macro will expand to the name of the vehicle when it comes time to request a new bridge. If the newly received ping is from the node named *gilda*, the bridge request from the above pattern may look like:

```
PSHARE_CMD = cmd=output, src_name=DEPLOY_GILDA, dest_name=DEPLOY,  
              route=2.56.111.1:9200
```

Note the vehicle name in the MOOS variable macro was expanded to be upper case, even though the ping information referred to the vehicle as *gilda*. This is just to aid in the convention that MOOS variable are typically all upper case. If one really want a literal expansion with no case altering, the macro `$v`, lower-case `v`, may be used instead. The macro is only respected as part of `src` field. In other words, if the bridge were configured with `alias=DEPLOY_$V`, the macro would not be expanded. The other type of macro implemented is the `$N` macro, as in:

```
bridge = src=LOITER_$N, alias=LOITER
```

The `$N` macro will expand to the integer value representing number of unique pings received thus far. For example, if three pings are received and accepted from *henry*, *james*, and *ike*, `uFldShoreBroker` would make three separate posts, perhaps looking like:

```
PSHARE_CMD = "src_name=LOITER_1, dest_name=LOITER, route=2.56.111.1:9200"  
PSHARE_CMD = "src_name=LOITER_2, dest_name=LOITER, route=2.56.111.4:9200"  
PSHARE_CMD = "src_name=LOITER_3, dest_name=LOITER, route=2.56.111.12:9200"
```

This may be useful when used in conjunction with another MOOS process generating output generically for N vehicles, without having to know the vehicle names in advance.

2.5 A Common Configuration Shortcut - the `qbridge` Parameter

A common usage pattern is to configure `uFldShoreBroker` to request two types of bridges for a given variable, for example:

```
bridge = src=DEPLOY_ALL, alias=DEPLOY  
bridge = src=DEPLOY_$V, alias=DEPLOY  
bridge = src=RETURN_ALL, alias=RETURN  
bridge = src=RETURN_$V, alias=RETURN
```

This could be use in the shoreside community for easily commanding vehicles. When the user wishes to deploy all vehicles, a posting of `DEPLOY_ALL="true"` does the trick. If the user wishes only the vehicle *james* to return, a posting of `RETURN_JAMES="true"` may be made. This pattern is so common that this shortcut is supported. This is done with the `qbridge`, "quick bridge", parameter. The above four configuration lines could be accomplished instead by:

```
qbridge = DEPLOY, RETURN
```

3 Usage Scenarios for the `uFldShoreBroker` Utility

The `uFldShoreBroker` was designed with a canonical command-and control scenario in mind. The idea is that the N deployed vehicles have a common autonomy protocol implemented. For example, a message to deploy or return a vehicle is the same message for each deployed vehicle. The idea is that two types of communication channels need to be established with `pShare`, (a) messages sent to all vehicles, and (b) messages sent to a particular named vehicle. The convention proposed here is to do this with the two types of bridging described in the discussion of the `qbridge` parameter, in Section 2.5. For a variable such as `DEPLOY`, a posting in the shoreside community to `DEPLOY_ALL` would go to all known vehicles, and a posting to `DEPLOY_HENRY` would only go to that particular vehicle.

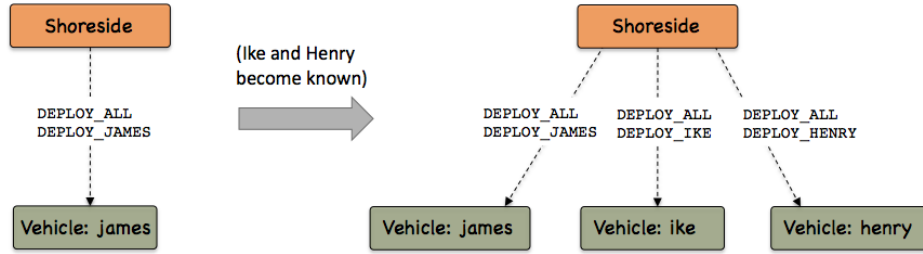


Figure 2: **Common uFldShoreBroker Usage Scenario:** As vehicles become known to the shoreside, each vehicle has two new bridges established. The first is the same for all vehicles to allow broadcasting, and the second bridge is unique to the particular vehicle, for individual command and control.

4 Terminal and AppCast Output

The `uFldShoreBroker` application produces some useful information to the terminal on every iteration of the application. An example is shown in Listing 1 below. This application is also appcast enabled, meaning its reports are published to the `MOOSDB` and viewable from any `uMAC` application or `pMarineViewer`. See the appcasting documentation for more on appcasting and viewing appcasts.

On line 1, the application iteration is shown, more as a heartbeat indicator. In lines 4-7, the primary variables consumed and posted by `uFldShoreBroker` are summarized in terms of how many posts have been made and received for each variable.

Listing 4.1: Example terminal output of the `uFldShoreBroker` tool.

```

1  =====
2  uFldShoreBroker_PS shoreside                               (109)
3  =====
4
5  Total PHI_HOST_INFO    received: 11
6  Total NODE_BROKER_PING received: 180
7  Total NODE_BROKER_ACK   posted: 180
8  Total PSHARE_CMD        posted: 34
9
10
11 =====
12      Shoreside Node(s) Information:
13 =====
14
15      Community: shoreside
16      HostIP: 128.30.27.202
17      Port MOOSDB: 9000
18      Time Warp: 6
19      IRoutes: localhost:9200
20
21 =====
22      Vehicle Node Information:
23 =====
24
25 Node   IP           Elap   pShare
26 Name   Address        Status  Time   Input Route(s)      Skew
27 -----
28 henry  128.30.27.202    ok      0.0   128.30.27.202:9301  1.6542

```

```

29 gilda 128.30.27.202 ok      0.0  128.30.27.202:9302  1.7572
30
31 Recent Events (2):
32 [22.06]: New node discovered: gilda
33 [16.04]: New node discovered: henry

```

In lines 11-19, the key shoreside properties are listed. Typically, but not always, the shoreside community is name "shoreside" as indicated on line 15. The shoreside IP address, determined by `pHostInfo`, is shown on line 16. The time warp, and `MOOSDB` port are read from the shoreside `.moos` file and listed on lines 17 and 18. The input routes used by `pShare` are listed on line 19. If lines 16 or 19 are blank, `uFldShoreBroker` will not make any connections and the first place to look is whether or not `pHostInfo` is running and producing valid information.

In lines 21-33, the status of each of the known vehicles is shown. The first two vehicles had their pings accepted. Their IP addresses are shown in the second column. Their status is shown in the third column. The elapsed time in the fourth column is the time since the last ping was received by the shoreside. The fifth column shows the input routes being used by `pShare` running on the vehicle node. If multiple routes are in use, this will be shown over multiple lines. The sixth column shows the time skew between the timestamp in the `NODE.BROKER.PING` message compared to the time it was received. Some of this is due to (a) latency in transmission, (b) latency due to App Ticks in brokers on both sides, and (c) clock discrepancy between the shoreside and the node computers. It's also worth mentioning that the skew will be magnified for higher time warps. Currently incoming ping connection requests are not denied due to a high clock skew, but this may be implemented in the future.

5 Configuration Parameters of uFldShoreBroker

The following parameters are defined for `uFldShoreBroker`.

Listing 5.2: Configuration Parameters for uFldShoreBroker.

<code>auto_bridge_appcast:</code>	Suppress the normal automatic bridging of the <code>APPCAST_REQ</code> variable. The default is false.
<code>auto_bridge_mhash:</code>	Suppress the normal automatic bridging of the <code>MISSION_HASH</code> variable. The default is false.
<code>auto_bridge_realmcst:</code>	Suppress the normal automatic bridging of the <code>REALMCAST_REQ</code> variable. The default is false.
<code>bridge:</code>	Names a MOOS variable to be bridged to a node community. Section 2.3.
<code>keyword:</code>	Optionally set a keyword. If set, incoming <code>NODE.BROKER.PING</code> messages must contain this keyword or else they ping will not get a response. Section TBD.
<code>qbridge:</code>	Shorthand notation for a common bridging pattern. Section 2.5.
<code>try_vnode:</code>	TBD
<code>warning_on_stale:</code>	If true, generate a warning if a previously known node has not been heard from in more than 10 seconds.

As an example, `bridge = src =DEPLOY_ALL, alias=DEPLOY`, will result in the bridging of variable

`DEPLOY_ALL` from the local `MOOSDB`, to the variable `DEPLOY` in a remote MOOS community. Further examples are given in Section 2.

An Example MOOS Configuration Block

Listing 3 shows an example MOOS configuration block produced from the following command line invocation:

```
$ uFldShoreBroker --example or -e
```

Listing 5.3: Example configuration of the `uFldShoreBroker` application.

```
1  =====
2  uFldShoreBroker Example MOOS Configuration
3  =====
4
5  ProcessConfig = uFldShoreBroker
6  {
7      AppTick    = 4
8      CommsTick  = 4
9
10     warning_on_stale    = false (default)
11     auto_bridge_realmcst = true  (default)
12     auto_bridge_appcast = true  (default)
13     auto_bridge_mhash   = true  (default)
14
15     bridge = src=DEPLOY_ALL, alias=DEPLOY
16     bridge = src=DEPLOY_$V, alias=DEPLOY
17
18     try_vnode = 192.168.4.24:9200
19     try_vnode = 192.168.4.25:9200
20
21     qbridge = RETURN
22     qbridge = NODE_REPORT, STATION_KEEP
23
24     bridge = src=UP_LOITER_$N, alias=UP_LOITER
25
26     // Note: [qbridge = FOO] is shorthand for
27     //         [bridge = src=FOO_$V, alias=FOO] and
28     //         [bridge = src=FOO_ALL, alias=FOO]
29
30     app_logging = true // false is default
31 }
```

6 Publications and Subscriptions for `uFldShoreBroker`

The interface for `uFldShoreBroker`, in terms of publications and subscriptions, is described below. This same information may also be obtained from the terminal with:

```
$ uFldShoreBroker --interface or -i
```

6.1 Variables Published by uFldShoreBroker

The primary output of `uFldShoreBroker` to the `MOOSDB` are the requests to `pShare` for registrations, and the outgoing acknowledgment replies to remote node/vehicle communities.

- **APPCAST**: Contains an appcast report identical to the terminal output. Appcasts are posted only after an appcast request is received from an appcast viewing utility. Section 4.
- **PMB_REGISTER**: A message to `pShare` to add a new bridge for a given variable and given target MOOS community at a specified IP address and port number.
- **NODE_BROKER_ACK**: A message written locally but bridged to a remote vehicle MOOS community, containing IP address and port information about the local shoreside community.

6.2 MOOS Variables Subscribed for by uFldShoreBroker

The `uFldShoreBroker` application subscribes to the following MOOS variables:

- **APPCAST_REQ**: A request to generate and post a new appcast report, with reporting criteria, and expiration.
- **PHI_HOST_INFO**: Information about the local host IP address, the MOOS community name, the port on which the DB is running, and the port on which the local `pShare` is listening for UDP messages.
- **NODE_BROKER_PING**: Information published presumably by `uFldNodeBroker` running in a remote vehicle community. Message has information about the node host including the community name, IP address, the port number for the `MOOSDB`, input route(s) for the local `pShare` process.

6.3 Command Line Usage of uFldShoreBroker

The `uFldShoreBroker` application is typically launched with `pAntler`, along with a group of other shoreside modules. However, it may be launched separately from the command line. The command line options may be shown by typing:

```
$ uFldShoreBroker --help or -h
```

Listing 6.4: Command line usage for the `uFldShoreBroker` tool.

```
1 =====
2 Usage: uFldShoreBroker file.moos [OPTIONS]
3 =====
4
5 Options:
6   --alias=<ProcessName>
7       Launch uFldShoreBroker with the given
8       process name rather than uFldShoreBroker.
9   --example, -e
10      Display example MOOS configuration block.
```



```
11  --help, -h
12      Display this help message.
13  --interface, -i
14      Display MOOS publications and subscriptions.
15  --version, -v
16      Display release version of uFldShoreBroker.
```

uFldNodeComms: Simulating Inter-vehicle Communications

June 2018

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139

1	Overview	1
1.1	The Difference Between a Node Report and Node Message	2
2	Handling Node Reports	3
2.1	The Criteria for Routing Node Reports	3
2.2	Optional Limitation on the Node Report Share Rate	4
2.3	Node Report Transmissions and pShare	4
3	Asymmetric Node Report Sharing	5
3.1	Bestowing a Vehicle with an Enhanced Stealth Property	5
3.2	Bestowing a Vehicle with an Enhanced Listening Property	6
4	Handling Node Messages	6
4.1	The Criteria for Routing Node Messages	6
4.2	Enforcing a Minimum Time Between Node Messages	7
4.3	Enforcing a Maximum Node Message Length	7
4.4	Posting Messages to a Vehicle Group	7
5	Visual Artifacts for Rendering Inter-Vehicle Communications	8
6	Node Report Filtering and Sharing	9
7	Terminal and AppCast Output	10
8	Configuration Parameters of uFldNodeComms	11
9	Publications and Subscriptions for uFldNodeComms	13
9.1	Variables Published by uFldNodeComms	14
9.2	Variables Subscribed for by uFldNodeComms	14

1 Overview

The `uFldNodeComms` application is a tool for handling node reports and messages between vehicles. Rather than directly sending node reports and messages between vehicles, `uFldNodeComms` acts as an intermediary to conditionally pass a report or message on to another vehicle, where conditions may be the inter-vehicle range or other criteria. The assumption is that `uFldNodeComms` is running on a topside or shoreside computer, and receiving information about the present physical location of deployed vehicles through node reports. The typical layout is shown in Figure 1.

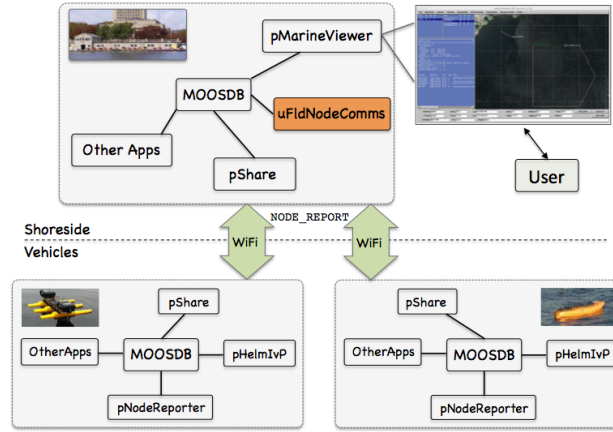


Figure 1: **Typical uFldNodeComms Topology:** A shoreside or topside community is receiving information from several deployed vehicles, in the form of node reports. The node reports contain time-stamped updated vehicle positions, from which the speed and distance measurements are derived and posted to the shoreside MOOSDB.

In short, `uFldNodeComms` subscribes for incoming node reports for any number of vehicles, and keeps the latest node report for each vehicle. On each iteration, for a each vehicle, if the node report has been updated, the report is published to a specially created MOOS variable for the other $n-1$ vehicles. A user-configured criteria is applied before publishing the new information. Typically this criteria involves the range between vehicles, but the criteria may be further involved. The idea is conveyed for three vehicles *alpha*, *bravo*, and *charlie*, shown below in Figure 2.

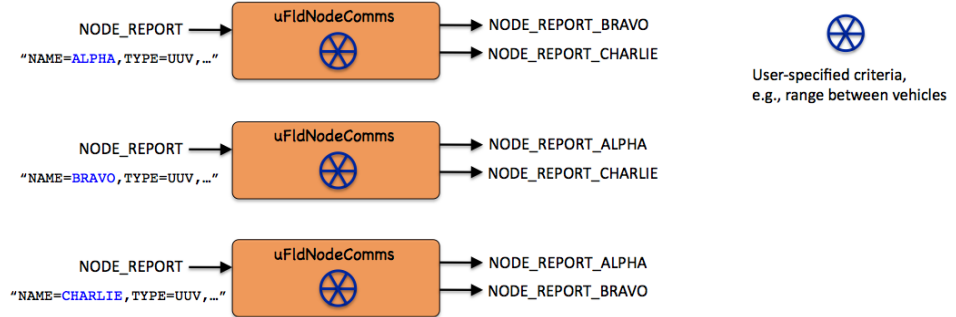


Figure 2: **Brokering by uFldNodeComms:** Each incoming node report is sent out to a specially named variable corresponding to one each of the other $n-1$ vehicles.

1.1 The Difference Between a Node Report and Node Message

There are two distinct message types in play with `uFldNodeComms`:

- Node reports - concise vehicle trajectory/state, in `NODE_REPORT`
- Node messages - open ended inter-vehicle content, in `NODE_MESSAGE`

A *node report* contains concise information regarding a vehicle's state, including its position, speed, heading and depth for underwater vehicles. It may also include information regarding the helm,

including the current all-stop state and helm mode. It is typically published by `pNodeReporter` running on each vehicle, and generated by default at 4Hz. This may be faster for higher speed vehicles operating in close proximity, and lower for slower moving vehicles operating in expansive waters. A node report is typically contained in one of two variables, either `NODE_REPORT_LOCAL` or `NODE_REPORT`. The former is used by `pNodeReporter` to indicate the report has been generated locally about ownship. When this information is sent off-board, to the shoreside, `uFldNodeComms`, and eventually other vehicles, it converted to the variable `NODE_REPORT`.

A *node message* contains open ended inter-vehicle message content, in the form of (a) a destination vehicle, (b) a MOOS variable name, and (c) the value of the MOOS variable. Not all messages sent will arrive at their destination and `uFldNodeComms` is the arbiter of which messages get through. There is no single app the generates a node message. Any app, by the configuration of the user, may publish an outgoing message to `NODE_MESSAGE_LOCAL`. This indicates the message was generated on ownship to be sent elsewhere. In the uField simulation scheme, using `uFldNodeComms`, it will arrive on the shoreside as `NODE_MESSAGE` and, if allowed through, will arrive on the destination vehicle(s) as `NODE_MESSAGE`. The receiving vehicle will unpack the incoming node message using an app called `uFldMessageHandler`.

In short, `uFldNodeComms` reasons about node reports and node messages, but it uses node reports to have situational awareness of all vehicle positions. It uses this situational awareness to be the arbiter of which node messages are allowed to travel between which vehicles, depending on the restrictions configured by the user as `uFldNodeComms` parameters.

2 Handling Node Reports

Node reports may contain a bundle of useful information for sharing between vehicles. Perhaps the most important is the present vehicle position and trajectory. This may be used by the receiving vehicle for collision avoidance and formation keeping and so on. The vehicle position is also used by `uFldNodeComms` to determine if the node reports themselves are to be shared between vehicles. The inter-vehicle ranges derived from the `NODE_REPORT` messages are also used to determine if other more generic information contained in the `NODE_MESSAGE` variable is to be shared between vehicles.

2.1 The Criteria for Routing Node Reports

The basic criteria for sharing node reports is range. By default, the node report received for any one vehicle is passed on to *all* other known vehicles within comms range of the originating vehicle. The comms range is set by the parameter `comms_range` as on line 9 in Listing 3. Starting with this as a baseline, a few other factors may be involved in determining whether a node report is shared.

Using Vehicle Group Information

If `uFldNodeComms` is configured with the parameter `groups` set to true, as on line 21 in Listing 3, then node reports are only shared between vehicles having the same group name. The group name is a field contained in the node report itself, so the onus is on the vehicle to include this information as part of its report. The `pNodeReporter` application contains an optional configuration parameter `group=<group-name>` where the group information is declared for inclusion in all node reports. The

motivation for the grouping option is to support multi-vehicle competitions where some vehicles want to convey positions to teammates, but not adversarial vehicles.

The `groups` feature only affects the passing of node reports between vehicles. It does not affect the passing of node *messages* discussed further below. Node messages contain their addressee information explicitly.

The `msg_groups` feature only affects the passing of node *messages* between vehicles. It does not affect the passing of node *reports* discussed above. This feature is introduced in the release following 19.8.2.

Establishing and Inter-Vehicle Critical Range

When the two vehicles are within a range deemed critical, as set by the `critical_range` configuration parameter as on line 10 in Listing 3, node reports are shared between vehicles regardless of the `comms_range` parameter and the `groups` parameter. The default for this parameter is 30 meters. The thought behind this feature is that, while it may be advantageous to not broadcast your own vehicle position to non group members for the purposes of a competition, it may be a good idea to share this information for the sake of collision avoidance.

Checking for Staleness

Since up-to-date inter-vehicle range information is used as part of the criteria in determining whether a vehicle receives a new node report from another, the position of the candidate recipient vehicle needs to reasonably up-to-date. The `stale_time` configuration parameter may be set as on line 11 in Listing 3, to determine the amount of elapsed time without receiving a node report from a vehicle before it is considered stale. If a recipient vehicle becomes stale, it will also not receive node messages.

Ignoring a Group

The user may specify a group name with the `ignore_group` parameter. All incoming node reports that match this group will simply be ignored. Introduced after Release 19.8.x.

2.2 Optional Limitation on the Node Report Share Rate

The `min_report_interval`, given in seconds, sets the minimum amount of time between node report messages from one vehicle to another. The default value is -1, disabling this feature.

Note that when the `view_node_rpt_pulses` parameter is set to true, then `uFldNodeComms` also publishes a visual artifact in a posting to `VIEW_COMMS_PULSE`. By limiting the node report share rate, the rate of this variable posting is limited too. This may also be a consideration in extreme missions of very high numbers of vehicles and very high time warp, for easing the burden on the viewer, e.g., `pMarineViewer`.

2.3 Node Report Transmissions and pShare

Node reports are communicated to recipient vehicles in coordination with the `pShare` application. For each unique vehicle name discovered by `uFldNodeComms` via received node reports, it will publish

a new MOOS variable `NODE_REPORT_NAME`. This variable will be published with the contents of other vehicles' node reports.

As shown in Figure 2, if three vehicles, *alpha*, *bravo*, and *charlie* become known, three corresponding MOOS variables `NODE_REPORT_ALPHA`, `NODE_REPORT_BRAVO`, and `NODE_REPORT_CHARLIE` will be published. The variable `NODE_REPORT_ALPHA` will be published with reports from *bravo* and *charlie* and so on. To make this happen, `uFldNodeComms` needs the corresponding share relationships from, for example, `NODE_REPORT_ALPHA` in the shoreside community to the variable `NODE_REPORT` in the alpha community on the alpha vehicle. This sharing relationship is established separately by the `uFldShoreBroker` application running in the shoreside community.

3 Asymmetric Node Report Sharing

Under normal circumstances, if one vehicle is within range to send a node report or message to another, it should also be able to receive node reports and messages from the same vehicle. In this section we discuss ways to make this relationship asymmetric. This is done in one of two ways:

- Increasing a vehicle's *stealth*, i.e., requiring other receiving vehicles to be closer than normal to receive reports and messages, or
- Increasing a vehicle's *earange*, i.e., allowing other receiving vehicles to be farther than normal and still receive node reports and messages.

3.1 Bestowing a Vehicle with an Enhanced Stealth Property

By using the *stealth* parameter, one vehicle may be given a bit of an advantage. The *stealth* parameter is assigned to a particular vehicle and is a number in the range [0.5, 1]. A message from source to destination is sent if:

$$actual_range(src, dest) < comms_range * stealth(src) \quad (1)$$

By default, the *stealth* factor for each vehicle is 1. A stealthy vehicle with a factor of one half, means the receiving vehicle needs to be twice as close as it would otherwise to receive the stealthy vehicle's node report or node message. The `critical_range` parameter may be used to override a vehicle's stealthiness with respect to sending node reports. That is, a message from source to destination is sent if:

$$actual_range(src, dest) < \max((comms_range * stealth(src)), critical_range) \quad (2)$$

The `stealth` value for a particular vehicle may be set in the MOOS configuration file as shown on line 18 in Listing 3. It may also be set dynamically by receiving mail on the variable `UNC_STEALTH`. For example the posting

```
UNC_STEALTH = vname=alpha, stealth=0.75
```

would immediately reset the *stealth* factor for vehicle *alpha* on the next iteration. The motivation for exposing this parameter via incoming MOOS mail is that another shoreside application, monitoring vehicle speed for example, may reward a vehicle operating in the field with increased *stealth* based on any criteria. This can be useful in constructing vehicle competitions.

3.2 Bestowing a Vehicle with an Enhanced Listening Property

In addition to the stealth property, a complementary property may be bestowed upon a vehicle using the `earange` parameter. The `earange` parameter is assigned to a particular vehicle and is a number in the range $[1, 2]$. A message from source to destination is sent if:

$$actual_range(src, dest) < comms_range * earange(dest) \quad (3)$$

By default, the `earange` factor for each vehicle is 1. A vehicle with a factor of two may receive node reports of another vehicle at twice the range it may otherwise. In the end the stealth and `earange` properties may cancel each other out when their extreme values are factored together. Taken together a message from source to destination is sent if:

$$actual_range(src, dest) < comms_range * stealth(src) * earange(dest) \quad (4)$$

The `earange` value for a particular vehicle may be set in the MOOS configuration file as shown on line 19 in Listing 3. It may also be set dynamically by receiving mail on the variable `UNC_EARANGE`. For example the posting

```
UNC_EARANGE = vname=alpha, earange=1.5
```

would immediately reset the `earange` factor for vehicle *alpha* on the next iteration. The motivation for exposing this parameter via incoming MOOS mail is that another shoreside application, monitoring vehicle speed for example, may reward a vehicle operating in the field with increased `earange` based on any criteria. This can be useful in constructing vehicle competitions.

4 Handling Node Messages

Node messages are of a generic structure for sharing a MOOS variable-value pair between a source node and a destination node. A node message from vehicle *alpha* to vehicle *bravo* would be posted locally by *alpha* with something like:

```
NODE_MESSAGE = "src_node=alpha,dest_node=bravo,var_name=FOOBAR,string_val=hello"
```

The local `NODE_MESSAGE` posting is shared to the shoreside community running `uFldNodeComms` where it is considered for re-routing to the destination vehicle.

4.1 The Criteria for Routing Node Messages

The basic criteria for sharing node messages is (a) the message addressee, and (b) the range between the source and destination vehicles. Since `uFldNodeComms` is receiving and keeping track of incoming node reports from all vehicles, it has ready access to the inter-vehicle range between the source and destination nodes. The same criteria used for sending node *reports* is used for sending node messages. It must meet the range criteria as perhaps modified by the stealth and `earange` factors discussed earlier. The only difference is that the `critical_range` parameter is irrelevant for the issue of sending node messages. This parameter was only used for node reports in the interest of safety and collision avoidance. There are however two other factors that may affect node message transmission, discussed next, message frequency and message size.

4.2 Enforcing a Minimum Time Between Node Messages

A maximum send frequency is enforced by requiring a minimum wait time between successful sends from a given source node. This minimum time is given by the parameter `min_msg_interval` as on line 13 in Listing 3. The default is 30 seconds. This interval is defined by the time starting with a successful transmission of a node message from a source to any destination. A separate log is kept by `uFldNodeComms` for each known vehicle.

Further clarification: Suppose for example, the `min_msg_interval` is set to 60 seconds. If a vehicle attempts an outgoing message 58 second after the previous message, it will not go through. If attempted 4 seconds later or 62 seconds after the previous successful message, then it will go through. It's as if the unsuccessful attempt never happened.

Continuing this example, consider a vehicle that attempts a message 62 seconds after the previous message was sent. Suppose this new message fails due to the inter-vehicle range being too large, the message length being too long, or the destination node not being recognized. In this case the failure results in another full 60 seconds being needed before the next message will be handled.

4.3 Enforcing a Maximum Node Message Length

The length of node messages may be limited with the parameter `max_msg_length`, as on line 14 in Listing 3. The default maximum length is 1000 characters. The length of a message refers to the number of characters in the `string_val` field. For example, the length of the message

```
NODE_MESSAGE = "src_node=alpha,dest_node=bravo,var_name=FOOBAR,string_val=hello"
```

is five. Limiting the message length is a proxy for inter-vehicle communications where message packet length is constrained, as with acoustic communications for example.

4.4 Posting Messages to a Vehicle Group

A node message is typically addressed to another named vehicle. The sender may also address the message by group name. All vehicles in the group meeting the prevailing range criteria will receive the message. The group associated with the vehicle is declared in the node report sent by that vehicle. A node message using a group address is similar except for the use of the `dest_group` parameter

```
NODE_MESSAGE = "src_node=alpha,dest_group=red_team,var_name=FOOBAR,string_val=hello"
```

The sender has the option of indicating *both* a group name and vehicle name as the message destination. It may also specify more than one vehicle name explicitly. Thus the following is allowed:

```
NODE_MESSAGE = "src_node=alpha,dest_name=bravo:charlie:gilda,dest_group=red_team,
var_name=FOOBAR,string_val=hello"
```

If a destination vehicle is specified twice in the list of destinations or implicitly in the named group, the message will be sent only once to the destination vehicle.

5 Visual Artifacts for Rendering Inter-Vehicle Communications

Each time a node report or message is sent to a vehicle by `uFldNodeComms`, another posting is made to the variable `VIEW_COMMS_PULSE`. This message may be subscribed for by another application using it to visually render the communications events. The screen shot in Figure 3 below shows four vehicles. The two bottom vehicles are sharing node reports indicated by the red and blue comms pulses.

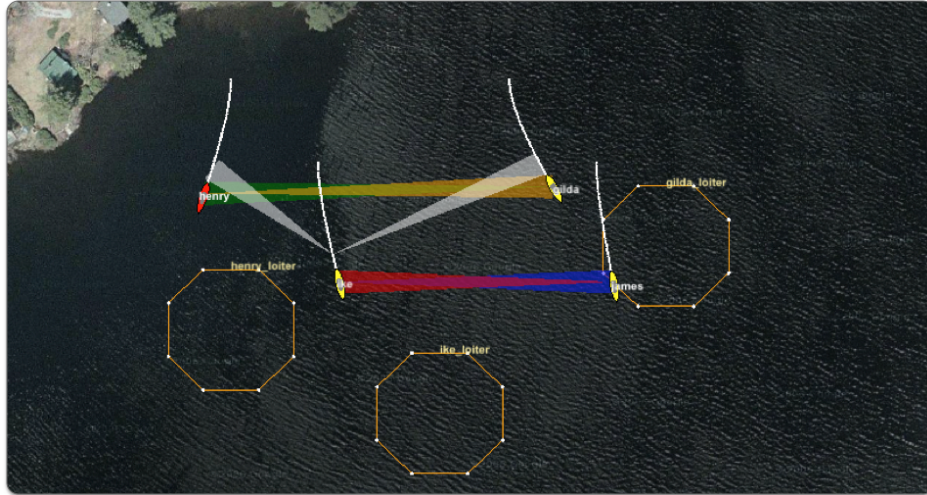


Figure 3: **Communication Pulses:** A visual artifact, a `VIEW_COMMS_PULSE`, is rendered between vehicles when either a node report or node message is generated. The white pulse indicates a node message, and the non-white pulses indicate a node report. The pulse widens from a point at the source vehicle to maximum width at the destination vehicle. A pulse will remain rendered in `pMarineViewer` for some number of seconds after initial post, unless it is replace by a new pulse with the same label.

The top two vehicles are also sharing node reports seen by the yellow and green pulses. The bottom left vehicle has also just sent a node message to the top two vehicles indicated by the two white pulse. Note the pair of white pulses were posted a few seconds prior to the present point in time since they point to vehicle positions just back in the vehicle history. The node reports are updated continuously, constantly replacing the pulse just previously posted.

The pulse colors are chosen automatically by `uFldNodeComms`. They may be toggled off in `pMarineViewer` with the '@' key. The comms pulse is conveyed in a posting to the variable `VIEW_COMMS_PULSE`. The format is shown with the following example.

```
VIEW_COMMS_PULSE = "sx=109.63,sy=-60.06,tx=30.96,ty=-60.22,beam_width=7,duration=10,
                    fill=0.35,label=JAMES2IKE,edge_color=green,fill_color=red,
                    time=2652414456.59,edge_size=1"
```

Comms pulses may be generated by other applications besides `uFldNodeComms`, and may be consumed and rendered by other applications besides `pMarineViewer`. The definition of the comms pulse object and the methods for serializing and de-serializing between object and string representation may be found in the `lib_geometry` library in the `moos-ivp` software tree.

6 Node Report Filtering and Sharing

As discussed above, a core input to `uFldNodeComms` are all the node reports arriving from each vehicle. Node reports are needed to:

- Send node reports back out to other vehicles, serving as proxy AIS system between vehicles
- Determine where vehicles are relative to each other, to decide whether inter-vehicle messages are sent from one vehicle to another.

The rate of node report arrivals may be huge, especially when the shoreside is supporting a multi-vehicle mission of say 50 or more vehicles, and in simulations of say time warp 50 or higher. On each vehicle a node report is typically generated by `pNodeReporter` at 4Hz. Together this would amount to 10,000 node reports per second. The `uFldNodeComms` app can routinely handle this load without issue on any machine we have tested.

However, `pMarineViewer` also ingests this same stream of node reports, primarily for the purpose of updating and rendering vehicle trajectories. In a high time warp simulation resulting in a single vehicle generating a node report at 100-200 node reports per second, this rate is far more than what `pMarineViewer` needs to smoothly render a vehicle trajectory. Roughly 10-20 position updates per second is sufficient to give the human eye the feeling of a smoothly updated trajectory.

As an optional configuration, `uFldNodeComms` can be configured to re-post node reports under a different variable name, `NODE_REPORT_UNC`. The re-posting is generated at a rate that considers the prevailing time warp, and ensures a posting rate commensurate with what is needed for human visual updates, e.g. 10-20Hz. The idea is conveyed below in Figure 4.

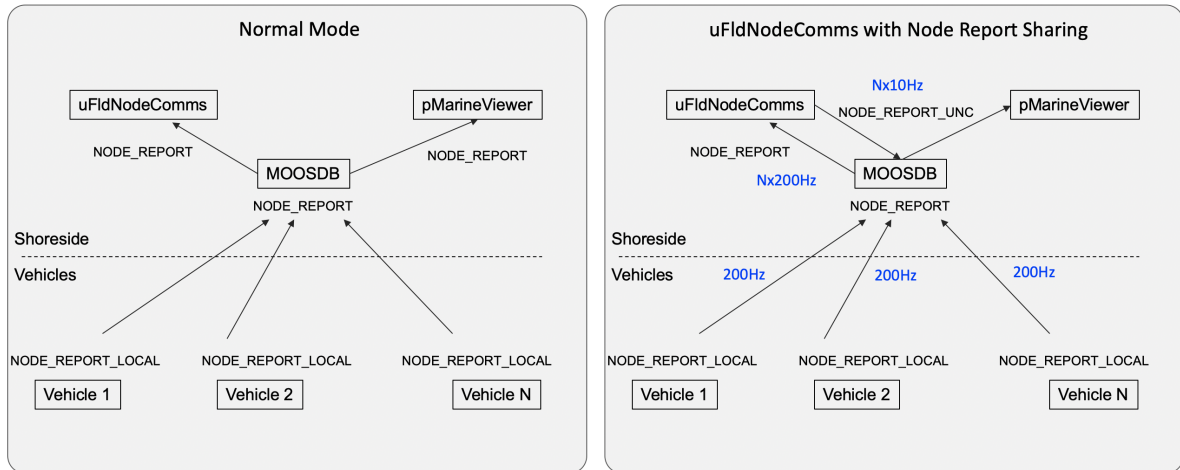


Figure 4: **Node Report Sharing:** With the optionally node report sharing configured, `uFldNodeComms` can act as a filter of node reports, re-posting node reports at a slower rate, in very high time warp situations.

An experienced MOOS user may note that `pMarineViewer` could be configured to simply register for `NODE_REPORT` at a slower rate, when the app registers for this variable. Of course the registration rate would have to also consider the prevailing time warp, but this is easy enough. This approach

would be problematic however because using the MOOSDB to filter out say 9 of every 10 node reports may result in un-even filtering per vehicle. Since all vehicles publish to `NODE.REPORT` and not `NODE.REPORT.ABE` for example, the above approach may end up filtering out 99 of 100 reports from one vehicle and 4 of 5 reports from another, leading to jumpy rendering and false stale report warnings.

By shifting node report filtering to `uFldNodeComms`, under such extreme high contact, high time warp situations, `uFldNodeComms` shifts computational burden away from `pMarineViewer`. The viewer is probably the most computationally burdened app running in these kinds of missions and `uFldNodeComms` is already processing the full set of node reports anyway as part of its basic job. Finally, most users are typically using a computer with multiple cores available, with `uFldNodeComms` and `pMarineViewer` running on different cores.

7 Terminal and AppCast Output

The `uFldNodeComms` application produces some useful information to the terminal and identical content through appcasting. An example is shown in Listing 1 below. On line 2, the name of the local community (usually shoreside) is listed on the left. On the right, "0/0(1822) indicates there are no configuration or run warnings, and the current iteration of `uFldNodeComms` is 1822. Lines 4-13 show the configuration requested from the mission configuration file.

Lines 15-30 convey a summary of received and sent node reports from each vehicle. The number in parentheses at the end of lines 18-22 indicate the elapsed time since the last node report was received. The number in brackets indicates the rate of received node reports (total node reports received per vehicle over time since app startup). This number should roughly equal the AppTick for `pNodeReporter` running on each vehicle.

Listing 7.1: Example uFldNodeComms console and appcast output.

```

1  =====
2  uFldNodeComms shoreside                                0/0(1822)
3  =====
4  Configuration:
5      Comms Range: 1000
6      Critical Range: 10
7      Stale Drop: 5
8      Stale Foget: 100
9      Max Message Len: 1000
10     Min Message Int: 20
11     Min Report Int: 60
12     Apply Group: false
13     Share Reports: true (0.1)
14
15  Node Report Summary
16  =====
17     Total Received: 9112      Elapsed      Rate
18         ABE: 1828          (0.0)        [3.60]
19         BEN: 1824          (0.0)        [3.59]
20         CAL: 1820          (0.0)        [3.58]
21         DEB: 1818          (0.0)        [3.58]
22         EVE: 1822          (0.0)        [3.58]

```

```

23      -----
24      Total Sent: 90
25          ABE: 27
26          BEN: 18
27          DEB: 18
28          EVE: 27
29      -----
30      Stale Vehicles: 0
31
32      Node Message Summary
33      =====
34      Total Msgs Received: 3
35          ABE: 3      (24.9)
36      -----
37          Total Sent: 3
38          BEN: 3
39      -----
40      Total Blocked Msgs: 0
41          Invalid: 0
42          Missing Info: 0
43          Stale Receiver: 0
44          Too Recent: 0
45          Msg Too Long: 0
46          Range Too Far: 0
47
48      =====
49      Most Recent Events (3):
50      =====
51      [96.22]: Msg rec'd: src_node=abe,dest_node=ben,var_name=UPDATE_LOITER,string_val=speed=2.4
52      [71.10]: Msg rec'd: src_node=abe,dest_node=ben,var_name=UPDATE_LOITER,string_val=speed=2.4
53      [56.46]: Msg rec'd: src_node=abe,dest_node=ben,var_name=UPDATE_LOITER,string_val=speed=2.4

```

The summary of node messages is shown in the second half of the report, in lines 32-46 in this case. The total messages received is shown on line 34, with a breakdown of where they have been received from in the following lines. Starting on line 37, a summary of sent node messages is given. First the total sent messages on line 37 and a breakdown of receivers in the following lines. A summary of blocked messages is given next, in this case in lines 40-46. The total number of blocked messages is given first, followed by the possible reasons for blocking in lines 41-46. Finally, the most recent messages are shown as events in the last lines of the report.

8 Configuration Parameters of uFldNodeComms

The following parameters are defined for `uFldNodeComms`.

Listing 8.2: Configuration Parameters for `uFldNodeComms`.

`comms_range`: Max range outside which inter-vehicle node reports and node messages will not be sent. Legal values: any numerical value. The default is 100 meters. Section 2.1.

<code>critical_range:</code>	Range in meters within which inter-vehicle node reports will be shared even if group membership would otherwise disallow. Legal values: any numerical value. The default is 30 meters. Section 2.1.
<code>debug:</code>	If true, further debugging information is produced to the terminal output. Legal values: true, false. The default is false.
<code>earange:</code>	A parameter in the range of [1,10] for extending the range a vehicle may otherwise hear node reports from other vehicles. The default is 1. Section 3.2.
<code>groups:</code>	If true, inter-vehicle node reports are shared only if two vehicles are in the same group. May be overridden if the two vehicles are within the critical range. The vehicle group designation is typically declared in <code>pNodeReporter</code> . The default is false. Section 2.1.
<code>ignore_group:</code>	If specified, incoming node reports that match this group will be ignored. Introduced after Release 19.8.x.
<code>min_msg_interval:</code>	The number of seconds required between messaged sends for any one source vehicle. The default is 30 seconds. Section 4.2.
<code>max_msg_length:</code>	The total number of characters that may be sent in a string component of a node message. The default is 1000. Section 4.3.
<code>min_rpt_interval:</code>	The number of seconds required between report sends for any one source vehicle. The default is -1, meaning no dropping of reports. Section 2.2.
<code>msg_groups:</code>	If true, inter-vehicle node messages are shared only if the two vehicles are in the same group. The vehicle group designation is typically declared in <code>pNodeReporter</code> . The default is false. Section 2.1.
<code>msg_color:</code>	The color of inter-vehicle message comms pulses. The default is "white". Section 5.
<code>msg_repeat_color:</code>	The color of inter-vehicle message comms pulses when the message represents a repeat of a previously dropped message, under mediated message passing. The default is "light_green". Section 5.
<code>stealth:</code>	A parameter in the range [0,1] for reducing the range other vehicles may otherwise hear the node reports from a source vehicle. The default is 1. Section 3.1.
<code>pulse_duration:</code>	A duration, in seconds, that the comms or node pulse should be viewable. It is a field in <code>VIEW_COMMS_PULSE</code> message that is respected by at least <code>pMarineViewer</code> to render and slowly fade-out the rendering after this duration. The default is 10 seconds. Section 5.
<code>shared_node_reports:</code>	When enabled with true, node reports are re-posted to the shoreside community as <code>NODE_REPORT_UNC</code> . The default is false. This feature may also be more commonly enabled via incoming message. See <code>UNC_SHARED_NODE_REPORTS</code> . Section 6.
<code>stale_time:</code>	Time in seconds after which a vehicle will not receive node reports or messages unless a node report has been received by that vehicle. The default is 5 seconds. Section 2.1.

- `verbose`: If true, status reports are displayed to the terminal during operation. The default is false.
- `view_node_rpt_pulses`: If true, comms pulses are rendered between vehicles whenever a node report successfully makes its way from one vehicle to another. The default is true. Section 5.

An Example MOOS Configuration Block

Listing 3 shows an example MOOS configuration block produced from the following command line invocation:

```
$ uFldNodeComms --example or -e
```

Listing 8.3: Example configuration of the `uFldNodeComms` application.

```

1  =====
2  uFldNodeComms Example MOOS Configuration
3  =====
4
5  ProcessConfig = uFldNodeComms
6  {
7      AppTick    = 4
8      CommsTick  = 4
9
10     comms_range    = 100          // default (in meters)
11     critical_range = 30          // default (in meters)
12     stale_time     = 5           // default (in seconds)
13
14     max_msg_length = 1000        // default (# of characters)
15     min_msg_interval = 30        // default (in seconds)
16     min_rpt_interval = -1        // default (in seconds)
17
18     verbose        = true         // default
19     groups         = false        // default
20     msg_groups      = false        // default
21
22     stealth        = vname=alpha, stealth=0.8
23     earange        = vname=alpha, earange=4.5
24
25     shared_node_reports = true/false or any non-neg number
26
27     pulse_duration = 10;          // default (in seconds)
28     view_node_rpt_pulses = true   // default
29 }
```

9 Publications and Subscriptions for `uFldNodeComms`

The interface for `uFldNodeComms`, in terms of publications and subscriptions, is described below. This same information may also be obtained from the terminal with:


```
$ uFldNodeComms --interface or -i
```

9.1 Variables Published by uFldNodeComms

The primary output of `uFldNodeComms` to the MOOSDB are the node reports and node messages out to the recipient vehicles, and visual artifacts to be used for rendering the inter-vehicle communications.

- **APPCAST**: Contains an appcast report identical to the terminal output. Appcasts are posted only after an appcast request is received from an appcast viewing utility. Section 7.
- **NODE_MESSAGE_<VNAME>**: Node messages destined to be sent to a destination vehicle <VNAME> indicated as the recipient in the node message.
- **NODE_REPORT_<VNAME>**: Node reports destined to be sent to a given vehicle <VNAME> about the vehicle named in the node report.
- **NODE_REPORT_UNC**: When node report sharing is enabled, a subset of all node reports will be re-posted to this variable. See Section 6.
- **VIEW_COMMS_PULSE**: A visual artifact for rendering the sending of a node report or node message between vehicles.

9.2 Variables Subscribed for by uFldNodeComms

The `uFldNodeComms` application subscribes to the following MOOS variables:

- **APPCAST_REQ**: A request to generate and post a new appcast report, with reporting criteria, and expiration.
- **NODE_MESSAGE**: A node message from one vehicle to another.
- **NODE_REPORT**: A node report for a given vehicle from `pNodeReporter`.
- **NODE_REPORT_LOCAL**: Another name for a node report for a given vehicle from `pNodeReporter`.
- **UNC_STEALTH**: The extra stealth allowed a given vehicle to "hide" node reports to others.
- **UNC_EARANGE**: The extra range allowed a given vehicle to "hear" node reports of others.
- **UNC_FULL_REPORT_REQ**: For a given vehicle name, the next node report from each of the other vehicles will be received, regardless of all other criteria such as range, frequency, group. This only applies for the very next report only, then all criteria reverts. This is a convenient debugging tool.
- **UNC_SHARED_NODE_REPORTS**: If enabled, set to true, `uFldNodeComms` will re-post node reports from all vehicles to the variable **NODE_REPORT_UNC** but will clamp the publication rate, regardless of time warp, to a rate suitable for visual consumption, typically no more than 10 updates per second per vehicle. This feature is used in coordination with `pMarineViewer` to improve performance in high time warp missions with a high number of contacts. This variable is typically published by `pMarineViewer` requesting this service from `uFldNodeComms`. Section 6.
- **UNC_VIEW_NODE_RPT_PULSES**: A way for external apps to tell `uFldNodeComms` to turn off or on the rendering of comms pulses whenever a node report is sent from one vehicle to another. Section 5.

Command Line Usage of uFldNodeComms

The `uFldNodeComms` application is typically launched with `pAntler`, along with a group of other shoreside modules. However, it may be launched separately from the command line. The command line options may be shown by typing:

```
$ uFldNodeComms --help or -h
```

Listing 9.4: Command line usage for the `uFldNodeComms` tool.

```
1  =====
2  Usage: uFldNodeComms file.moos [OPTIONS]
3  =====
4
5  Options:
6    --alias=<ProcessName>
7        Launch uFldNodeComms with the given process
8        name rather than uFldNodeComms.
9    --example, -e
10       Display example MOOS configuration block.
11    --help, -h
12       Display this help message.
13    --interface, -i
14       Display MOOS publications and subscriptions.
15    --version, -v
16       Display the release version of uFldNodeComms.
17
18  Note: If argv[2] does not otherwise match a known option,
19        then it will be interpreted as a run alias. This is
20        to support pAntler launching conventions.
```